

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Институт заочного образования

09.03.01 "Информатика и вычислительная техника"
профиль "Программное обеспечение средств
вычислительной техники и автоматизированных систем"

Кафедра прикладной математики и кибернетики

Лабораторная работа 2
Алгоритмы и вычислительные методы оптимизации
Нахождение начального плана транспортной задачи

Вариант 9

Выполнил:

студент гр.ЗП-31

_____/Ушакова Е. А./
ФИО студента

«__» _____ 2026 г.

Проверил

доцент каф. ПМиК

_____/Галкина М.Ю./

«__» _____ 2026 г.

Оценка _____

Новосибирск 2026 г.

Задание:

Написать программу, находящую начальный опорный план транспортной задачи одним из указанных методов

0. Метод северо-западного угла.

Матрицу тарифов, запасы поставщиков и потребности потребителей вводить из файла.

Программа должна уметь работать как с открытой, так и закрытой моделью транспортной задачи.

Вывести распределение перевозок на каждом шаге метода и затраты на перевозки в получившемся решении.

Описание используемого метода.

Заполнение таблицы начинается с верхней левой клетки («северо-западный угол»). На каждом шаге для заполнения выбирается 84 клетка, находящаяся справа или снизу от предыдущей. Начальный опорный план в этом методе обычно получается далеким от оптимального, т.к. не учитывается стоимость перевозок.

Исходный текст программы на языке Си.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
int main () {
```

```
int *a;
```

```
int *b;
```

```
int **c;
```

```
int **x;
```

```
int rows, cols;
```

```
printf("Найти начальный опорный план транспортной задачи методом северо-западного  
угла.\n\n");
```

```
FILE *file = NULL;
```

```
file = fopen("data.txt", "r");
```

```
fscanf(file, "%d %d", &rows, &cols);
```

```
//processing A, B
```

```
a = malloc(rows * sizeof(int));
```

```
b = malloc(cols * sizeof(int));
```

```
int sumA = 0;
```

```
int sumB = 0;
```

```
for(int i = 0; i < rows; i++) {
```

```
fscanf(file, "%d", &a[i]);
```

```

sumA += a[i];
}
for(int i = 0; i < rows; i++) {
fscanf(file, "%d", &b[i]);
sumB += b[i];
}

c = (int **)malloc(rows * sizeof(int *));
for (int i = 0; i < rows; i++) {
c[i] = (int *)malloc(cols * sizeof(int));
}
for (int i = 0; i < rows; i++){
for (int j = 0; j < cols; j++) {
fscanf(file, "%d", &c[i][j]);
}
}

printf("Потребители: %d \nПоставщики: %d\n\n", sumA, sumB);

if (sumA != sumB) {
if (sumA > sumB) {
cols++;
b = realloc(b, cols * sizeof(int));
if ((b == NULL)) {
printf("memory allocation error\n");
return 0;
}
b[cols - 1] = sumA - sumB;
} else {
rows++;
a = realloc(a, rows * sizeof(int));
if ((a == NULL)) {
printf("memory allocation error\n");
return 0;
}
a[rows - 1] = sumB - sumA;
}
c = (int **)realloc(c, rows * sizeof(int *));
if ((c == NULL)) {
printf("memory allocation error\n");
return 0;
}
for (int i = 0; i < rows; i++) {
c[i] = (int *)realloc(c[i], cols * sizeof(int));
if ((c[i] == NULL)) {
printf("memory allocation error\n");
return 0;
}
}
}
}

```

```
x = (int **)malloc(rows * sizeof(int *));
for (int i = 0; i < rows; i++) {
x[i] = (int *)malloc(cols * sizeof(int));
}
```

```
for(int i = 0; i < rows; i++)
{
for(int j = 0; j < cols; j++)
{
x[i][j] = -1;
}
}
```

```
fclose(file);
printf("Исходные данные, приведенные к закрытой модели: \n");
printf("A: ");
for(int i = 0; i < rows; i++) {
printf("%d ", a[i]);
}
printf("\nB: ");
for(int i = 0; i < cols; i++) {
printf("%d ", b[i]);
}
printf("\n");
printf("C:\n ");
for (int i = 0; i < rows; i++){
for (int j = 0; j < cols; j++) {
printf("%2d ", c[i][j]);
}
printf("\n ");
}
```

```
printf("\n-----\n");
```

```
int iter = 0;
for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
if (b[j] == 0) continue;
if (a[i] == 0) break;
if (a[i] >= b[j]) {
x[i][j] = b[j];
a[i] -= b[j];
if(a[i] == 0 && i != rows - 1) {
x[i + 1][j] = 0;
}
b[j] = 0;
} else {
x[i][j] = a[i];
b[j] -= a[i];
if(b[j] == 0 && j != cols - 1) {
x[i][j + 1] = 0;
}
```

```

}
a[i] = 0;
}
iter++;
printf("Шаг %d: \n", iter);
for (int i = 0; i < rows; i++){
for (int j = 0; j < cols; j++) {
printf("%2d ", x[i][j]);
}
printf("\n");
}
printf("-----\n");
}
}

```

```

printf("\nОпорный план транспортной задачи:\n\n");
for (int i = 0; i < rows; i++){
for (int j = 0; j < cols; j++) {
printf("%2d ", x[i][j]);
}
printf("\n");
}
printf("-----\n");

```

```

int z = 0;

```

```

for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
z += c[i][j] * x[i][j];
}
}

```

```

printf("Затраты на перевозки в получившемся решении: %d\n", z);

```

```

return 0;
}

```

Результат работы программы:

```
irbi42@irbi42-HONOR-X14:~/uni/aivmo$ ./lab2
```

Найти начальный опорный план транспортной задачи методом северо-западного угла.

Потребители: 55

Поставщики: 60

Исходные данные, приведенные к закрытой модели:

A: 10 15 20 10 5

B: 10 10 20 20

C:

23 3 4 1

2 4 3 1

1 2 3 1

3 3 1 1

0 0 0 0

Шаг 1:

10 -1 -1 -1

0 -1 -1 -1

-1 -1 -1 -1

-1 -1 -1 -1

-1 -1 -1 -1

Шаг 2:

10 -1 -1 -1

0 10 -1 -1

-1 -1 -1 -1

-1 -1 -1 -1

-1 -1 -1 -1

Шаг 3:

10 -1 -1 -1

0 10 5 -1

-1 -1 -1 -1

-1 -1 -1 -1

-1 -1 -1 -1

Шаг 4:

```
10 -1 -1 -1
  0 10  5 -1
-1 -1 15 -1
-1 -1 -1 -1
-1 -1 -1 -1
```

Шаг 5:

```
10 -1 -1 -1
  0 10  5 -1
-1 -1 15  5
-1 -1 -1 -1
-1 -1 -1 -1
```

Шаг 6:

```
10 -1 -1 -1
  0 10  5 -1
-1 -1 15  5
-1 -1 -1 10
-1 -1 -1 -1
```

Шаг 7:

```
10 -1 -1 -1
  0 10  5 -1
-1 -1 15  5
-1 -1 -1 10
-1 -1 -1  5
```

Опорный план транспортной задачи:

```
10 -1 -1 -1
  0 10  5 -1
-1 -1 15  5
-1 -1 -1 10
-1 -1 -1  5
```

Затраты на перевозки в получившемся решении: 326